

BSA Capital

A marketplace for close-ended private credit funds on the XRP Ledger

Youssef Jeddi, Stan Stelcher

XRPL Lending Protocol Hackathon 2026, Paris

Developer feedback, 13 September 2026

A PR was opened for documentation correction : <https://github.com/XRPLF/XRPL-Standards/pull/634>

1. What cost us time

1.1 The two-party LoanSet signature. 114 minutes.

This is the protocol's central transaction and we could not produce one. Twenty-seven submissions.

A counterparty signs over the prefix 0x43505400, not the ordinary 0x53545800. In beta.0 the SDK helper omitted it, so every attempt returned Counterparty: Invalid signature. We found the answer by reading the wallet extension's source code.

Three signals pointed the wrong way. Omitting the signature returns temBAD_SIGNER with text about duplicate signers, which reads as a multisign problem. The intuitive signing order returns "must be first signed by first party", so the order is learnable only by trial. And combineLoanSetCounterpartySigners sits beside the signing helper mirroring the batch pair, so sign-then-combine looks natural; it is multisign only and throws.

One worked example would have removed all of it. beta.1 shipped the fix during the event and it works.

1.2 An unserviceable loan schedule is accepted at origination

LoanSet has no maturity field. You give an interval and a count, and maturity is their product, so anyone wanting "mature on this date" divides backwards. The bound that matters is the vault's RedemptionDate, which no LoanSet field mentions. Pick the numbers directly, as every short example does, and the loan matures four minutes after origination.

LoanPay then returns tecEXPIRED permanently. No extend, no renegotiate. The principal stays in the borrower's account with no route back to the vault that lent it, so those depositors can never be paid.

We wrote this bug three times in one codebase, twice in the same file. Our unit test on the schedule helper passed throughout: the helper was correct, nothing asserted that the call sites used it.

The fund-of-funds case is worse. The borrower puts every drop into sub-funds, so it holds no cash until those redeem. With more than one instalment the first falls due inside exactly that window. Not late, impossible. XLS-66 offers no bullet or interest-only shape.

1.3 The validity window closes during a two-party handoff

A WalletConnect session binds to one account, so a two-party LoanSet must be parked while the operator switches accounts. Autofill sets LastLedgerSequence to current plus 20, about 80 seconds, which is less than a human needs.

The field is covered by the first signature, so it cannot be widened afterwards. It has to be set wide before anyone signs, and nothing near the counterparty material says so.

1.4 tecNO_AUTH covers two independent gates

Receiving vault shares needs domain membership and the recipient's own MPTokenAuthorize. Both refusals are tecNO_AUTH, the fixes differ, and the code does not say which applies. One occasion cost 656 seconds.

Every earlier flow created the holder's token implicitly during VaultDeposit, so the first place this appears is a secondary purchase.

The opt-in is also permissionless. We tested it deliberately: an uncredentialed buyer opted in successfully and was refused only at payment. Holding the token is therefore not evidence of eligibility, which matters for anything compliance-shaped. Separately, CredentialCreate returns tesSUCCESS while the subject is still refused until they accept.

1.5 Nothing can be enumerated from the ledger

account_objects needs an owner you already know, ledger_entry needs a 64-character id, and a depositor arriving at a platform has neither. The first useful screen of any XLS-65 product therefore cannot be built from

the ledger. We shipped a table, three endpoints and a client-side join that reads phase and price live while the database holds identity only.

The same gap exists one level down. A LoanBroker carries no loan list and Loans belong to the borrower, so a fund cannot enumerate its own loan book.

1.6 The lifecycle cannot be read off the explorer

Per-transaction verification is easy and we use it: every claim in our README links a Devnet transaction, ten of them. The lifecycle is what does not work. Phase is not a field. The Vault entry holds SubscriptionDate and RedemptionDate as Ripple-epoch integers and nothing derived, so the phase exists only as a comparison against ledger close time that something has to perform. With 1.5 on top, nobody can open an explorer, find our vault and read which phase it is in. We wrote that view ourselves: a phase badge, a lifecycle rail and a clock reading ledger close time rather than Date.now(), since the two disagree at exactly the boundary you want to demonstrate. About 130 lines to show what the ledger already determines.

2. Smaller frictions

What we saw	What it was	What would have helped
temINVALID	GracePeriod below an undocumented 60 second floor, bisected at 30, 59, 60.	The SDK checks grace against the interval but enforces no floor. Name the field and give the bound.
"is an illegal amount"	PeriodicPayment is fractional drops; an XRP amount must be whole.	Document the unit or expose a payable figure. We round up so we never underpay.
temDISABLED	OfferCreate against an MPT, with and without a DomainID.	It contradicts IsfMPTCanTrade being set on the issuance, which is what made us try.
no error	A fully repaid Loan survives with every balance field absent.	Delete it or add a settled flag. The entry resolving does not mean money is owed; ours reported settled loans as live.
no error	Non-compliant XLS-89 metadata is a console warning, invisible in a browser build.	asset_subclass is required when the class is rwa, icon is required and desc is not. Neither rule is in the type.
not applicable	No message signing in the wallet namespace.	Proving account control to your own backend means signing an AccountSet that is never submitted.

3. What we found running the implementation ourselves

We also ran our own node and audited XLS-65 and XLS-66 against it. Two findings we think are worth acting on.

- **A stranger can break a loan broker for 10 drops.** Create a credential naming the broker's own protocol account as subject. Every broker operation then returns tecINVARIANT_FAILED, the ledger declaring its own rules violated. The victims are the broker owner and a borrower who already repaid and can now close neither the loan nor their own account. Neither transacted with the attacker. The guard exists in the code but sits behind an amendment the lending dependency list does not require.
- **A vault cancels an issuer's transfer fee.** Waiving the fee inside a vault is correct, since moving your own money into a container is not a transfer. But VaultWithdraw accepts an optional Destination, so the waived path also delivers to a third party: 1000 delivered costs 2000 directly and 1000 through a vault, with nothing burned. The waiver follows the code path rather than the destination. The AMM equivalent has no destination field.

Four further defects are not vulnerabilities and share one shape, code that appears to enforce something and does not: a Data field accepted on a loan the object has no room for; an immutability list guarding a field that does not exist while omitting the one that does; a compile-time assert stating the opposite of the rule it protects; and two paths asking the same question where the strict answer is compiled out of the release build.

On method, since that is what makes such a list usable. Every entry states how we know it, either measured on the node with a control differing in one variable, or read in the source and never run. Three entries failed our own attacker-and-victim test during review and were moved out rather than kept. Eleven corrections are logged, several against our own severity claims, and one finding was withdrawn after re-checking. Coverage is 50 of 68 pairs, with the 18 gaps named. Vulnerability detail went to the mentors privately before this report.

4. What we would like

- A maturity date on LoanSet, or a refusal when the implied maturity falls before the vault's own term. This single change removes the most expensive mistake we made.
- A bullet or interest-only loan shape, for a borrower whose capital is illiquid until a known date.
- Separate result codes for the two receive gates, or metadata saying which one refused.
- A documented pattern for multi-party signing, stating that LastLedgerSequence must be widened before the first signature.
- A way to page vaults and loan brokers. Every XLS-65 product ships an indexer before it can draw a list.
- Message signing in the wallet namespace, so authentication stops presenting as a transaction approval.
- SDK validators that match what the ledger enforces, starting with the grace floor, and a metadata check that fails rather than warns.

5. What worked well

- **The three-phase model is legible and its errors are good.** tecT00_S00N on a withdrawal mid-term is clear enough that we now use it deliberately in the demo to show shares are locked. We built the lifecycle from the spec without a surprise.
- **VaultKind, SubscriptionDate and RedemptionDate needed no explanation, and the validator is better than we expected.** validateVaultCreate refuses a close-ended vault with dates missing, and an out-of-range kind, before anything reaches the network. The one gap is ours as much as the SDK's: it exports the enum VaultKind { vaultKindOpen, vaultKindClosed } then types the field number, so an editor never offers it and all ten of our call sites hold a literal 1 and a comment. WithdrawalPolicy is the same shape, and there 9 is accepted.
- **Compliance gating is native, and it is why the product exists.** A credential, a permissioned domain and a private vault refuse an uncredentialed depositor with tecN0_AUTH, and none of our code performs that check.
- **Cash-basis interest behaves exactly as described.** One LoanPay moved a vault from 60000000 to 60000005 drops with share supply unchanged, stepping price per share from 1 to 1.0000000833333333. Interest raises assets without minting shares, and the step is discrete.
- **Immutable phase dates are the right call.** We asked for editable dates, found VaultSet refuses them, and changed our minds: immutability is what makes the phase guarantee worth anything. Worth one line in the reference so teams do not design around editing them.
- **Once the shape is right it is very reliable.** Ten vault and broker transactions per seeding run, and seven permissioned domains, all accepted first try. Everything that cost us time cost it once.

6. A note on the event tooling

The capture hook reads any shell output containing a result code as a failed transaction. A sed of a React component was recorded as VaultCreate tecT00_S00N, a module-resolution error as a LoanBrokerSet failure, and reading a source file produced LoanSet tecEXPIRED plus a phantom retry when no LoanSet had been submitted.

It grows with the codebase, because the more the code mentions ledger codes in comments and UI strings, the more false failures appear. Any error or retry rate taken from the raw rows is inflated. We checked every figure in this report against the transcript individually.